SYNAP Threat Model

Threat Model

This document identifies the abuse vectors most relevant to a synthetic data generation system aimed at regulated sectors (finance, insurance, healthcare — see `PROJECT_NARRATIVE.md`), what this project currently does about each one, and what it deliberately does not (yet) address. It complements `SECURITY.md` (how to report a vulnerability) and `ROADMAP.md` (why certain gaps exist) rather than repeating them.

This is a living document, not a certification. A clean threat model is not a compliance guarantee — see §7.

---

1. Assets

What this system handles that an adversary might want, and why each one matters specifically in a regulated-sector context:

Asset	Why it matters
Anchor data (`anchor_csv`)	Real records used to build the Statistical Contract (Module 1). If it's customer or patient data, this is the asset every downstream control in this document exists to protect.
Real holdout (`real_holdout_csv`)	Real records never used for generation, held back specifically to test the *output* — equally sensitive, and often forgotten as "just a test set."
Training reference (`training_reference_csv`)	Same sensitivity as the anchor — it's used only for the Membership Inference Attack check, but it is real data sitting on disk regardless.
The generated dataset, before Module 7/8 clear it	Not yet proven safe. Treating a mid-pipeline dataset as already "synthetic and therefore safe" is itself a threat vector (see §3.1).
The schema and business rules	Can itself be sensitive — a schema can encode proprietary risk-scoring logic (e.g., the exact thresholds a lender uses to classify risk), independent of any data.
The local web server's job store (`synap.webserver.JOBS` , `JOB_WORKDIR`)	Holds uploaded files and generated outputs for the lifetime of the process — a local, temporary, but real copy of the same sensitive assets above.

---

2. Adversary Models

Four distinct adversaries, because "an attacker" is not one thing:

1. **A downstream consumer of the output** — someone who receives `dataset_final.csv` and the two PDF reports, with no access to the real data, the schema, or the running system. Their

only attack surface is the synthetic data itself.

2. **A malicious or careless insider with API/CLI access** — someone who can run `synap` or call `synap.webserver`'s API directly, with the real anchor/holdout files available locally. Their attack surface includes the system's own outputs (Contract statistics, diagnostics, audit reports) as an oracle, not just the final CSV.
3. **A co-tenant on the same machine** — someone with a local account on the same host running `synap --serve-site`, but without access to the anchor/holdout files themselves or the running process. Their attack surface is the filesystem (temp directories, output directories) and the loopback network port.
4. **A remote attacker if the local server is exposed beyond intent** — someone reaching `synap.webserver` over a network it was not designed for (e.g., through a misconfigured reverse proxy). This project's own mitigation is architectural (bind to loopback only — see §4.4); this adversary model exists specifically to state what happens if that assumption is violated by someone else's infrastructure choice.

---

3. Threats to the Statistical Output

3.1 — Treating unaudited output as safe

Vector: using `dataset_final.csv` before Module 7 has released it (or with `forcar_saida=True` silently set), because "the tool generated it" is mistaken for "the tool approved it."

Status: mitigated by design, not just by documentation. Module 7 blocks output by default without a real holdout; `forcar_saida` is recorded in `SynapResult.warnings` every time it's used, specifically so this can't happen silently. See `ARCHITECTURE.md` §9.

Residual risk: a human downstream of the pipeline can still discard or ignore the warnings. This is a process/governance risk, not a technical one this codebase can fully close — see §7.

3.2 — Membership inference beyond the tested adversary

Vector: Module 8's MIA check (`ARCHITECTURE.md` §10) tests one specific adversary: a distance-based attacker with access to `training_reference` and `real_holdout`. A real-world attacker with different or additional auxiliary data (e.g., a partial public record they already suspect belongs to someone in the training set) is a different, untested adversary.

Status: partially mitigated. An AUC near 0.5 against the tested attacker is real evidence, not nothing — but it is not a proof against every possible membership-inference strategy. `ROADMAP.md` §3.2 already names a stronger, shadow-model-based attacker as a considered (not yet built) improvement.

3.3 — Attribute inference from auxiliary knowledge

Vector: an adversary who already knows some of a real individual's attributes (e.g., age and rough location) uses the synthetic dataset's *statistical structure* — not any single row — to sharpen a guess about an attribute they don't know (e.g., a health condition correlated with the ones they do know), even without re-identifying a specific row.

Status: not tested by this project today. Module 8's privacy suite (DCR, NNDR, exact-match, MIA) is built to catch memorization and membership leakage — it does not currently measure attribute-inference risk, which is a related but distinct failure mode. Flagged here as a gap, not previously named in `ROADMAP.md`; tracking it there is a direct outcome of writing this document.

3.4 — Repeated queries as a statistical oracle

Vector: the Contract (Module 1) is built directly from the real anchor's exact mean, standard deviation, and correlation matrix — with no differential-privacy noise (see `ROADMAP.md` §3.1). An adversary with repeated API/CLI access, who can vary the schema or business rules slightly between calls and observe the resulting Contract, ASP pass/fail counts, or Fidelity Index each time, is in the same structural position as an attacker against any non-private statistical query interface: each response leaks a little more about the real underlying data than the last.

Why this matters specifically here: this project's own architecture document is explicit that Module 1's statistics are exact, not noised — this is not a hidden implementation detail, it is a stated design choice (`ARCHITECTURE.md` §2). A threat model's job is to say plainly what that choice means for an adversary with repeated access, not just for a one-shot user.

Status: not mitigated today. This is the single most consequential gap this document identifies, and it is the same gap `ROADMAP.md` §3.1 already names as "likely next" (a differentially private Contract) — this threat model is what makes that prioritization concrete rather than aspirational.

---

4. Threats to the System Itself

4.1 — CSV formula injection in exported output

Vector: if a schema's declared categories (or a generated numeric value formatted unexpectedly) begin with `=`, `+`, `-`, or `@`, a spreadsheet application (Excel, Google Sheets, LibreOffice Calc) opening `dataset_final.csv` may interpret that cell as a formula rather than data — a well-known class of vulnerability (CSV/formula injection) that has nothing to do with this project specifically but affects any tool that exports CSVs a human might later open in a spreadsheet.

Status: not currently mitigated. Neither `compilador.run_compilador` nor `synap.webserver`'s CSV export path currently escapes or quotes leading `=` / `+` / `-` / `@` characters in output cells. This is a concrete, addressable gap — tracked as a result of writing this document (see `ROADMAP.md`).

4.2 — Unbounded file upload size

Vector: `synap.webserver`'s `/api/generate` endpoint accepts multipart file uploads (`anchor_csv`, `real_holdout_csv`, `training_reference_csv`) with no `MAX_CONTENT_LENGTH` configured on the Flask app — a large upload is limited only by available disk and memory on the host running `synap --serve-site`.

Status: not mitigated. A local, single-user tool has a low practical severity for this (the "attacker" is the same person running the server), but it becomes real the moment `synap.webserver` is deployed anywhere more shared than a single laptop — which `ROADMAP.md` §5.2 already flags as out of today's intended deployment model for a different reason (no authentication). Both gaps point the same direction: don't expose `synap.webserver` beyond a trusted local user without addressing both.

4.3 — Unbounded concurrent generation jobs

Vector: every `POST /api/generate` call spawns a new background thread immediately, with no cap on concurrent jobs and no request rate limiting. Repeated calls (accidental, from a buggy client, or deliberate) can exhaust CPU and memory on the host.

Status: partially mitigated. `MAX_TARGET_ROWS_WEB` (5,000) bounds the *size* of any single job requested through the API, which limits the worst case per job — but does not limit how many jobs run at once.

4.4 — The local server is not a multi-tenant service

Vector: `synap.webserver` has no authentication and no per-user job isolation — any client that can reach the port can call `/api/generate` and can read any `job_id`'s results (`ROADMAP.md` §5.2 already documents this as a scope decision, not an oversight).

Status: mitigated by architecture, contingent on correct deployment. `serve_site` binds to `127.0.0.1` only — hardcoded, not a configurable CLI flag — specifically so this can't be loosened by accident from the command line. The residual risk is entirely in how someone else deploys it (e.g., a reverse proxy forwarding an external port to this loopback port without adding authentication of its own) — outside this codebase's ability to prevent, which is exactly why it's named here rather than left implicit.

4.5 — Job artifacts in a shared temporary directory

Vector: `JOB_WORKDIR` is `tempfile.gettempdir() / "synap_web_jobs"` — on a multi-user machine, the default permissions of the system temp directory determine whether another local account can read uploaded CSVs or generated outputs sitting there while a job exists or after it finishes (nothing currently cleans up completed job directories).

Status: not mitigated. No explicit permission-hardening (`os.chmod`) is applied to `JOB_WORKDIR` or its per-job subdirectories today.

4.6 — Anchor files are not filtered to declared columns

Vector: `neocortex`'s CSV loader (`pandas.read_csv`) reads every column in the provided anchor/holdout file, not only the columns named in the schema. If a real file contains extra sensitive columns beyond what the schema declares (e.g., a `customer_name` or national ID column alongside the `age` / `income` columns actually being modeled), those extra columns are loaded into memory as part of the working DataFrame, even though nothing in the pipeline is meant to use them.

Status: not mitigated. This is a defense-in-depth gap, not a demonstrated data leak — downstream modules build statistics only from schema-declared columns — but "the code only *uses* declared columns" is a weaker guarantee than "the code never *loads* undeclared columns at all," and a threat model should say so rather than rely on the distinction being obvious.

---

5. Threats from Misrepresentation (Not Purely Technical)

5.1 — Treating an empirical audit as a legal certification

Vector: Module 8's privacy approval is an empirical audit against specific, named tests (DCR, NNDR, exact-match, MIA) — not a legal determination that a dataset satisfies GDPR, HIPAA, or any other regulation. Presenting a "privacy: approved" verdict from this tool as regulatory compliance, to an auditor, regulator, or data-protection officer, would misrepresent what was actually measured.

Status: addressed in documentation, not something code can enforce. `SECURITY.md` already states this plainly ("the S.Y.N.A.P. is a tool to support that evaluation, not a substitute for it"). This threat model repeats it because misrepresentation of an empirical result as a legal one is a recognized failure mode specifically in the regulated-sector context this project targets — not a generic disclaimer.

5.2 — Survivorship bias from re-running until "approved"

Vector: because Module 7/8's verdicts depend on the random seed and the specific batch composition (see `EVALUATION.md` §7 for a measured example of seed-to-seed variation), a user could run the pipeline repeatedly with different seeds and keep only the run that happened to clear the fidelity/privacy/utility thresholds — reporting that one result as *the* result, rather than as one draw from a distribution.

Status: not preventable by the tool itself. `EVALUATION.md`'s own methodology (multiple seeds, reporting the spread, not just the best case) is the direct countermeasure this project applies to its own claims, and the same practice is the recommended mitigation for anyone using the tool operationally.

---

6. Summary Table

#	Vector	Adversary	Status
3.1	Unaudited output treated as safe	Downstream consumer	Mitigated by design (blocks by default)
3.2	MIA beyond the tested adversary	Insider / external	Partially mitigated (documented scope)
3.3	Attribute inference	External, with auxiliary data	**Not mitigated — newly tracked gap**
3.4	Repeated queries as a statistical oracle	Insider with API access	**Not mitigated — highest-priority gap**
4.1	CSV formula injection	Downstream consumer	**Not mitigated — newly tracked gap**
4.2	Unbounded upload size	Co-tenant / insider	Not mitigated (low severity at intended scale)
4.3	Unbounded concurrent jobs	Co-tenant / insider	Partially mitigated (per-job row cap only)
4.4	No auth / multi-tenancy	Remote, if exposed	Mitigated by architecture (loopback-only)
4.5	Shared temp directory permissions	Co-tenant	Not mitigated
4.6	Undeclared columns loaded from anchor file	Insider / co-tenant	Not mitigated (defense-in-depth gap)
5.1	Empirical audit misrepresented as legal compliance	Any	Documentation-level mitigation only
5.2	Survivorship bias across seeds	Insider (self-inflicted)	Not preventable by the tool; process mitigation only

Items marked **newly tracked gap** were identified while writing this document and were not previously listed in `ROADMAP.md` — they have been added there as a direct result (see `ROADMAP.md`'s changelog context in `CHANGELOG.md`).

---

7. What This Document Is Not

- **Not a penetration test.** No exploit was attempted against any vector listed here; each is a structural analysis of the codebase as written, not an empirical confirmation that it is exploitable in practice.
- **Not a compliance certification.** Nothing here should be quoted as evidence of GDPR, HIPAA, PCI-DSS, or any other regulatory conformance (see §5.1).
- **Not exhaustive.** This document covers the vectors most specific to a synthetic-data tool for regulated sectors — it does not restate generic infrastructure security (OS hardening, network segmentation, dependency-supply-chain auditing beyond what's noted in §4) that applies to any Python service equally.
- **A living document.** Report a vector not listed here the same way you'd report a vulnerability — see `SECURITY.md`. This document is expected to grow; a threat model that never changes is a sign no one is testing it against the actual system anymore.